Packet DeMultiplexer (PDM) Verification Architecture Document (VerDoc)

Scope: Testbench Architecture, Component Specifications, and Stimulus Generation

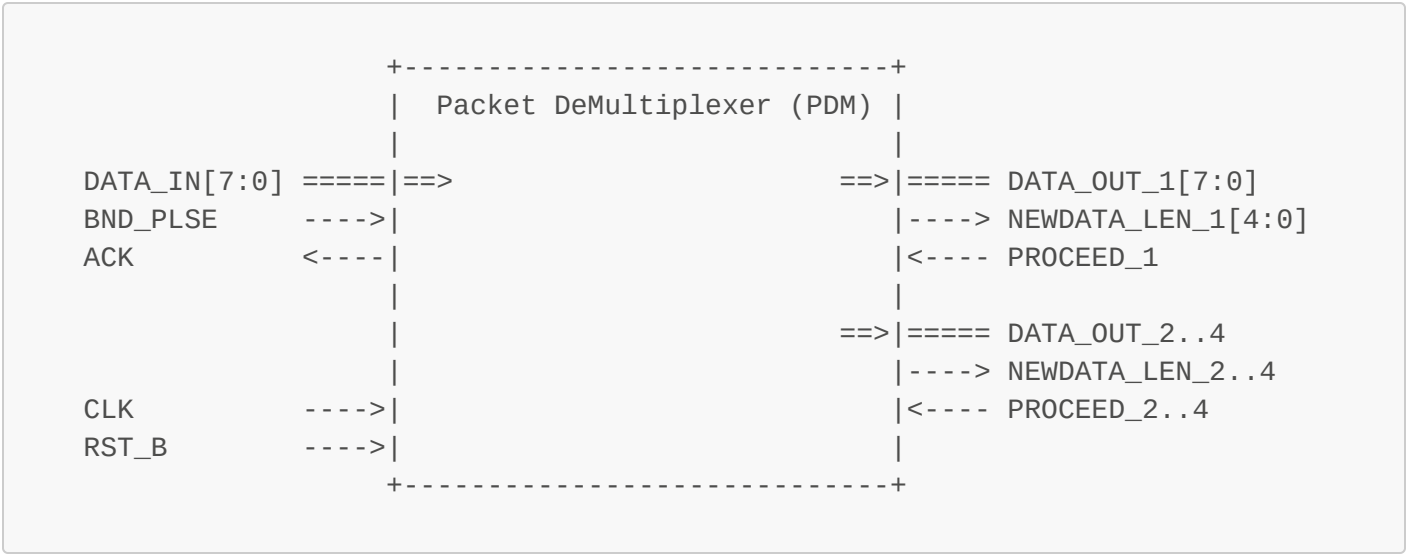
AI Disclaimer: AI was used to generate this verification document based on the lab requirements and a reference TB implementation, and is provided to help explain the advanced aspects of this testbench.

---

1. Introduction & Verification Scope

1.1 Device Under Test (DUT) Overview

The Packet DeMultiplexer (PDM) is a hardware switch designed to route unidirectional packet streams from a single upstream input interface to one of four downstream output interfaces (Ports 1 to 4).



- **System Signals**: Synchronous to `CLK` (50 MHz, 20 ns period) with an asynchronous active-low reset `RST_B`.
- **Input Interface**:
  - `DATA_IN[7:0]`: Carries packet data. Byte 0 is the destination port (`8'h01` to `8'h04`), followed by a payload of 4, 8, 12, or 16 bytes.
  - `BND_PLSE`: Active-high framing pulse asserted for exactly 1 cycle coincident with Byte 0 (start) and asserted again coincident with the final payload byte (end).
  - `ACK`: DUT response pulse asserted for 1 cycle within 1 to 4 clock cycles following the terminating `BND_PLSE`.
- **Output Interfaces (Ports 1 to 4)**:
  - `NEWDATA_LEN_x[4:0]`: DUT asserts a 1-cycle pulse indicating an incoming packet. The value represents the payload size (4, 8, 12, or 16 bytes).

- `PROCEED_x`: Downstream agent asserts a 1-cycle grant pulse within 1 to 4 cycles of detecting `NEWDATA_LEN_x`.
- `DATA_OUT_x[7:0]`: DUT streams the payload bytes starting exactly 1 clock cycle after `PROCEED_x` asserts, without interruption.

1.2 Verification Lab Scope

This verification architecture covers all components required to build a functional, reusable UVM testbench up through **stimulus generation**:

- **In Scope**:
  - Signal interfaces and clocking blocks.
  - Transaction items representing upstream and downstream traffic.
  - One Active Upstream Agent (driving stimulus packets and resets, monitoring protocol timing).
  - Four Reactive Downstream Agents (dynamically granting output transfers and capturing routed packets).
  - UVM Environment, Configuration Objects, and Synchronization mechanisms.
  - Test cases, including a directed sanity test.
- **Out of Scope (Future Verification Phase)**:
  - Predictor / Reference Model.
  - Scoreboard and Data Integrity Checkers.
  - Functional Coverage Collectors.

> [NOTE] Even though Scoreboard and Predictor components are deferred to a subsequent lab, the testbench monitors will expose standard UVM Analysis Ports (`upstream_txn_ap` and `downstream_txn_ap`), ready for the scoreboard connection later.

---

2. Testbench Architecture & Data Flows

2.1 System Architecture Diagram

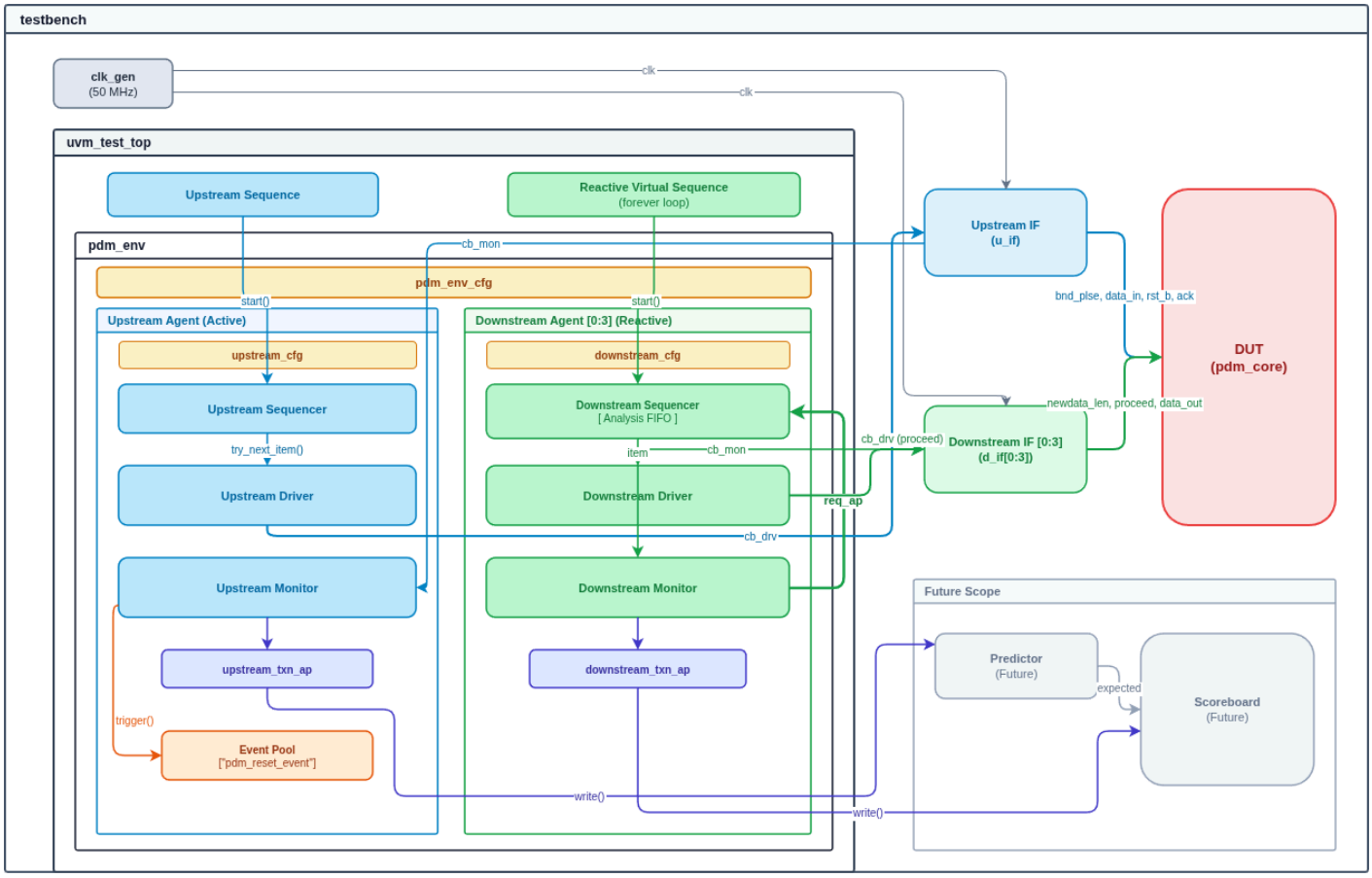


Figure 2.1: Packet DeMultiplexer (PDM) UVM Testbench Architecture and Data Flows.

2.2 High-Level UVM Hierarchy

The testbench follows standard UVM component containment:

```
testbench (top module)
 ├── clk_gen & dut_top (pdm_core)
 ├── Physical Interfaces (u_if, d_if[0:3])
 └── uvm_test_top (pdm_test_base / derived tests)
      └── pdm_env
           ├── upstream_agent (ACTIVE)
           │    ├── pdm_upstream_sequencer
           │    ├── pdm_upstream_driver
           │    └── pdm_upstream_monitor
           └── downstream_agent[0..3] (REACTIVE)
                ├── pdm_downstream_sequencer (with mon_req_fifo)
                ├── pdm_downstream_driver
                └── pdm_downstream_monitor
```

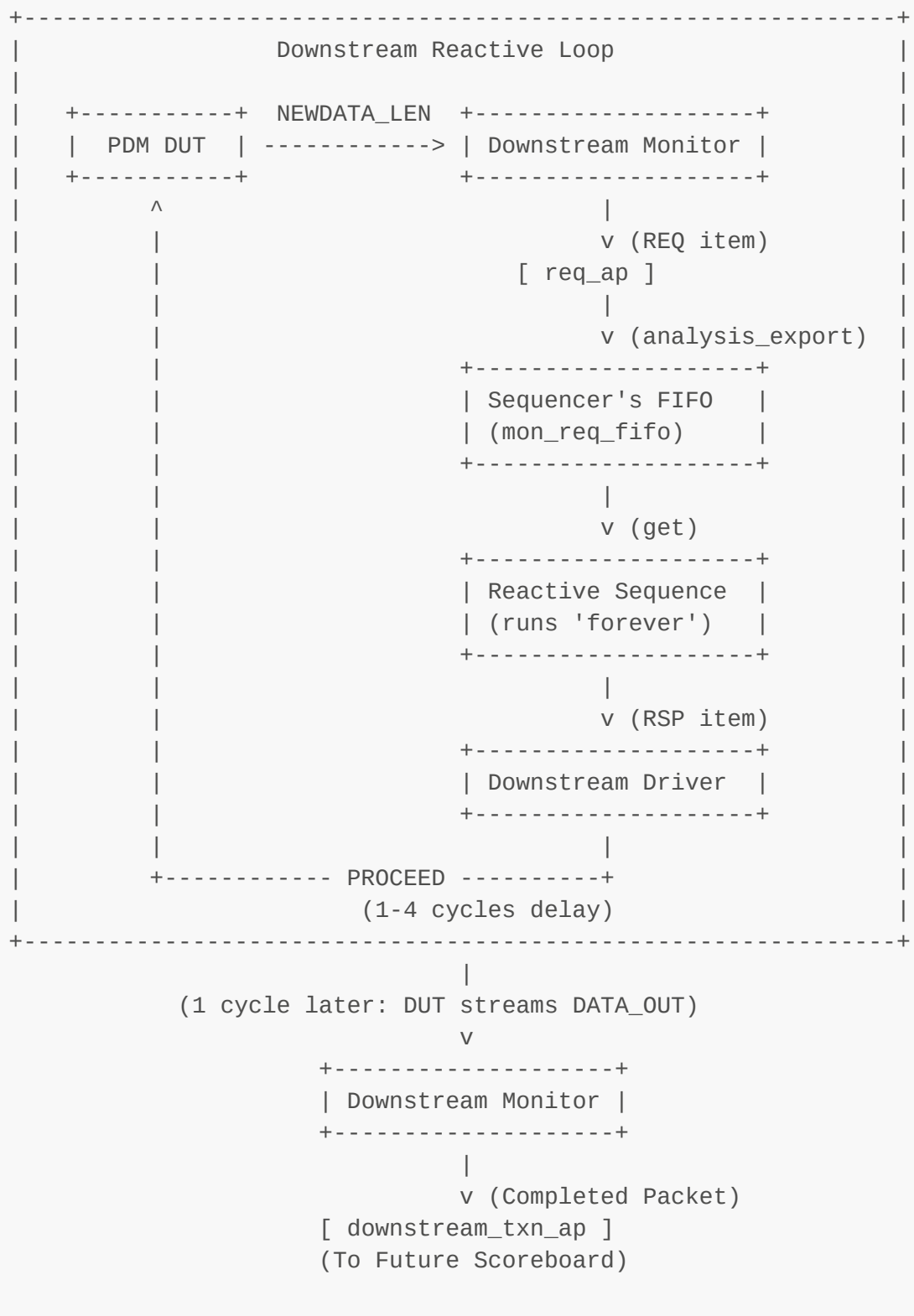
2.3 Active Upstream Data Flow

The upstream agent generates stimulus and drives the input interface actively:

```
[Upstream Sequence]
         |
         v (uvm_sequence_item)
[Upstream Sequencer]
         |
         v (seq_item_port)
[Upstream Driver] ----------> Drives bnd_plse, data_in, rst_b (via cb_drv)
                              Waits for ack
                                    |
                                    v
                                [PDM DUT]
                                    |
[Upstream Monitor] <---------- Samples bnd_plse, data_in, ack (via cb_mon)
         |
         v (reconstructed packet)
[upstream_txn_ap] (Analysis Port -> Future Predictor/Scoreboard)
```

2.4 Reactive Downstream Loop

Unlike the upstream interface, downstream interfaces cannot drive data at will. The downstream agent must act as a **reactive agent**: it remains dormant until the DUT announces incoming data, reacts within a protocol-defined window, and captures the resulting transmission.



Why a Dual-Role Monitor is necessary:

- In UVM, drivers should not passively sniff or decode complex protocol events on the bus; drivers only drive pins in response to sequence items.
- Monitors are dedicated, passive observers. The downstream monitor observes the bus and performs two vital functions:
    1. **Immediate Reaction Trigger**: Detects `NEWDATA_LEN_x` assertion and immediately writes a request item (`REQ`) to its analysis port (`req_ap`).
    2. **End-to-End Packet Collection**: Once `PROCEED_x` is asserted, it captures the subsequent payload bytes from `DATA_OUT_x`, validates length/timing, packages the complete packet (`RSP`), and writes it to `downstream_txn_ap`.

---

3. Configuration & Reset Architecture

3.1 Configuration Objects Hierarchy

To avoid hardcoding paths and parameters inside verification components, configuration is centralized using UVM configuration objects:

```
  pdm_env_cfg (top configuration)
   ├── pdm_upstream_agent_cfg   (contains vif, active/passive switch, etc.)
   └── pdm_downstream_agent_cfg [0..3] (contains vif, port_id, delay bounds,
etc.)
```

- **Environment Configuration (`pdm_env_cfg`)**: Created and populated at the test level. It instantiates one upstream agent configuration object and an array of four downstream agent configuration objects.
- **Agent Configurations (`pdm_upstream_agent_cfg`, `pdm_downstream_agent_cfg`)**: Pass component-specific settings (such as virtual interface handles, active/passive operating mode, output port identifiers `0..3`, timing constraints like min/max `PROCEED` delays, and handles to the agent's sequencer `sqr_h`) down to each agent.
- **Sequencer Handle Passing**: Each agent assigns its instantiated sequencer handle into its configuration object during `connect_phase()` (e.g., `cfg.sqr_h = sequencer`). Because top-level tests and virtual sequences maintain handles to these configuration objects, they can start sequences directly on `cfg.sqr_h` without relying on hierarchical paths (e.g., `env.upstream_agent.sequencer`).
- **Distribution**: The test stores `env_cfg` in `uvm_config_db`. The environment retrieves it in `build_phase()` and places each sub-configuration into `uvm_config_db` scoped to the corresponding agent.

3.2 Reset Awareness & Driver Architecture

Reset verification is a core requirement of this testbench. Resets can occur at initial power-on, predictably between transfers, or unpredictably mid-transfer.

1. Non-Blocking Driver Design

In standard UVM drivers, `seq_item_port.get_next_item(req)` is blocking. If a driver blocks waiting for a sequence item while an asynchronous reset occurs, or if an item is stuck in flight during reset, the driver can freeze or corrupt bus state.

Architectural Requirements:

- Drivers must remain clock- and reset-sensitive.
- Instead of blocking indefinitely on `get_next_item()`, drivers should use non-blocking retrieval mechanisms (such as `try_next_item()`) within a clock-driven loop:
  - Loop sensitivity: `@(posedge clk or negedge rst_b)`.
  - If `rst_b == 0`: immediately abort any active transaction, deassert bus signals, and clear driver state.
  - If `rst_b == 1`: process sequence items or advance the driver's state machine.
- *Alternative approach*: You may alternatively fork a dedicated reset-watcher process (`fork ... join_any` or `wait(!rst_b)`) that runs concurrently with the driver loop and terminates or resets the driving thread upon reset assertion.

2. Handling In-Flight Items During Reset

When an asynchronous reset drops mid-packet:

- The active item in progress must be terminated cleanly via `seq_item_port.item_done()` so the sequencer does not hang.
- Any items that were already fetched or queued during the reset window should be flushed (e.g., using `has_do_available()` and `get_next_item()` discards).

3. Global Reset Event Synchronization

When reset deasserts or drops asynchronously, other testbench components (such as monitors or scoreboards) need to know so they can clear temporary queues and reset their tracking logic.

- Use a global UVM event (e.g., `uvm_event_pool::get_global("pdm_reset_event")`) triggered by the monitor upon detecting reset.
- *Alternative approach*: You may alternatively use a dedicated reset analysis port and implement the write method on all components that need to be aware that a reset happened.

---

4. Component Design Specifications

4.1 Interface Architecture

- **`pdm_upstream_if`**
- **`pdm_downstream_if`**: Instantiated as an array of 4 interfaces corresponding to DUT output ports.
- **Clocking Blocks**

- **Asynchronous Reset Access**: Expose direct asynchronous access to `rst_b`.

4.2 Sequence Items (Transactions)

- **Upstream Item Requirements**:
  - CRV constraints should support keeping values within the specification while allowing constraints to be overridden or disabled for out-of-spec negative testing.
  - Must express operational intent: distinguishing standard data transfers from standalone reset pulses or mid-transfer reset injection.
- **Downstream Item Requirements**:
  - CRV constraints should support keeping values within the specification while allowing constraints to be overridden or disabled for out-of-spec negative testing.
  - Must distinguish between an initial notification request (`REQ`) and a completed packet response (`RSP`).

4.3 Upstream Agent Components

Upstream Driver

- **Role**: Translates sequence items into pin transitions on `pdm_upstream_if` and drives reset signals, follwing the DUT input protocol specification.

Upstream Monitor

- **Role**: Passively reconstructs upstream packets, monitors input protocol compliance, and triggers reset event when async reset is detected.

4.4 Downstream Agent Components

Downstream Sequencer

- **Role**: Receives incoming reaction requests using a `uvm_tlm_analysis_fifo` (`mon_req_fifo`) and arbitrates sequence items for the driver.

Downstream Driver

- **Role**: Drives reactive `PROCEED_x` on `pdm_downstream_if`.

Downstream Monitor

- **Role**: Observes output bus protocol and captures initial NEWDATA requests and completed payload packets.

---

5. Stimulus Generation

5.1 Reactive Sequence Execution

Unlike upstream stimulus sequences that generate a fixed count of packets and terminate, the downstream agent requires a continuously running reactive sequence that operates withing a `forever` loop and waits for REQ transactions using `mon_req_fifo.get(req)` before creating its RSP transaction.

5.2 Phase Objection Drain Time

- If all test objections drop immediately when upstream sequences finish, simulation will terminate prematurely, cutting off active downstream transfers.
- Configuring an objection drain time (e.g., via `set_drain_time`) on the run phase objection at the test level is one practical solution that ensures simulation remains active long enough for all in-flight downstream transactions to be fully transmitted, captured, and verified.
- *Note*: Once the predictor and scoreboard components are integrated in subsequent labs, end-of-test synchronization can be made dynamic (for example, raising objections in the scoreboard until expected transaction queues drain completely or synchronizing completion across components).